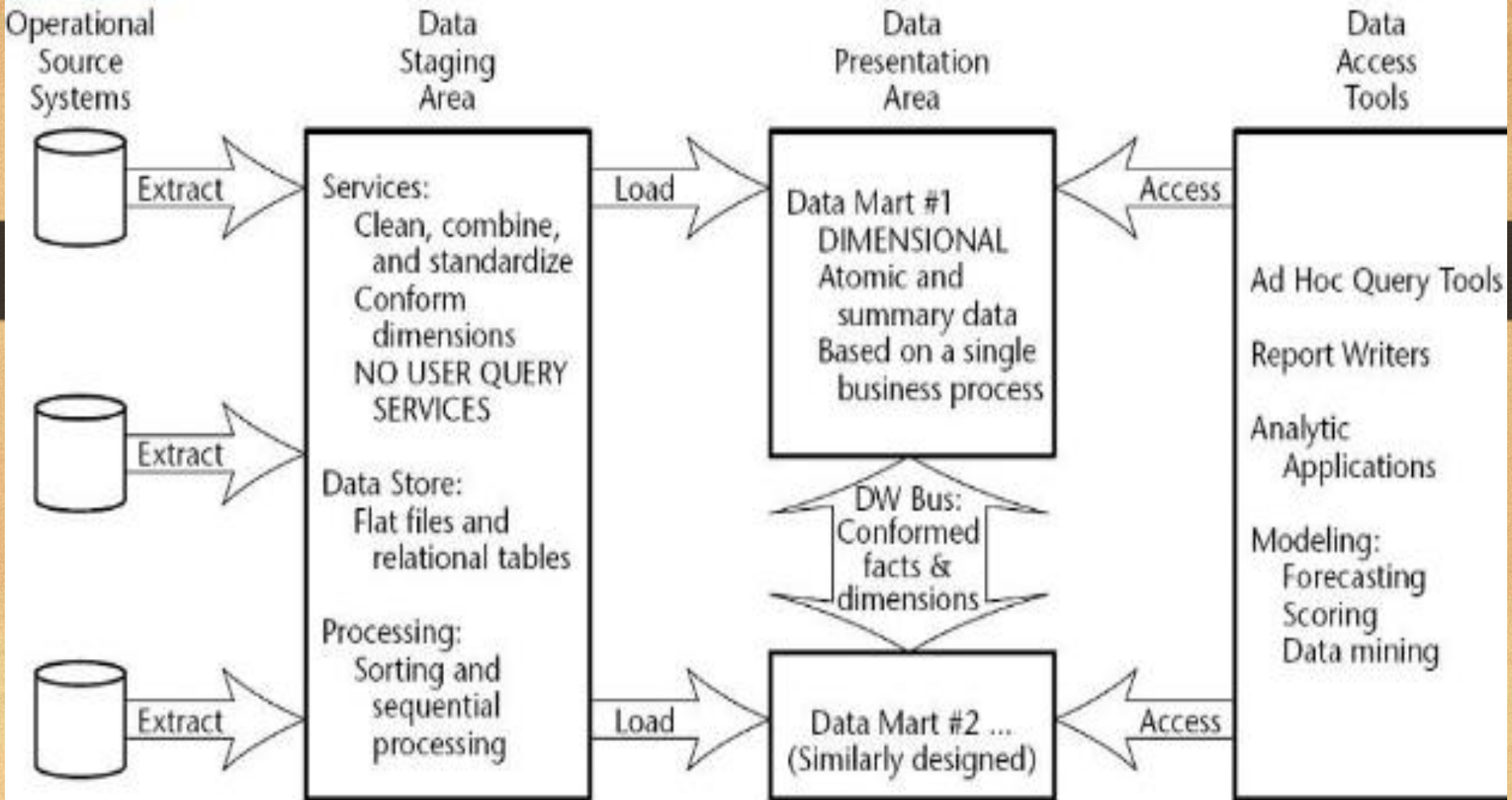


Components of a DW

DW: **Data Warehousing**



Components of a DW

- **Data sources.**
- **Data staging Area (DSA) / Operational Data Store (ODS)**
- **Data Marts/Data Warehouse.**
- **Presentation Area.**
- **Data Access Tool.**

DW Components

Operational Source Systems (Data Sources)

-
- These are the operational systems of record that capture the transactions of the business.
 - Should be thought of as outside the DW.
 - The main **priorities** of the sources systems are performance and availability.
 - The sources maintain little historical data. But IF you have a good data warehouse; **then no need for the data sources to hold history or at least to hold long time interval of historical data.**

Components of a DW

Data staging Area (DSA) / Operational Data Store (ODS)

- Is both a storage area and an operational area for a set of processes commonly referred as **Extract** → **Transform** → **Cleanse** → **Load** (**ETL**/ **ETCL**)
- The DSA/ODS is everything between the Data Marts/DW and the data presentation area / layer.

DW Components

Extraction (E)

-
- After setting up the business objectives of the DW or the Subject Statement.
 - Extraction is the first step in the process of getting data into the data warehouse environment.
 - Extraction means reading and understanding the source data and copying the data needed for the data warehouse into the staging area for further data manipulation.
 - Used methods for extraction:
 - Get the whole table every time.
 - Get the table incrementally (ONLY the new additions / Changes).
 - Using a fixed time periods.

DW Components

Extraction (E)

- Which method to use? **Different considerations**
 - **The Time Window taken to extract the data from the operational data sources:** the data extraction process should be completed in a certain amount of time. The less time it requires to do the data extraction, the better it is for a DW system. Because this means that DW will be updated more frequently.
 - **The second consideration is practically.** I.e., What are possible and what are not.
 - We need to extract data incrementally, but this may not be possible then we should search for and use another method.
 - We need to extract the whole table but if the table has 100 millions records, then we may have to do it incrementally (if possible)

DW Components

Extraction (E)

- **Whole Table Every Time?**

- The simplest way to download a table from an operational data source / source system into a data warehouse is to extract all records every time using **SELECT * FROM Datasource1.Table**
 - If a company has 500 **branch**
 - Each **branch** has a record and each record stores:
 - Name, Address and other attributes.
 - For 500 records, even if the **branch** table has **timestamp for both create and update**, it is probably quicker to get the entire table every time.

DW Components

Extraction (E)

- **Whole Table Every Time?**

- If we do a **SELECT** without a **WHERE** clause, the extraction process starts immediately/instantly. If the **WHERE** clause is filled by at least one condition e.g. Timestamp, the extraction process begins a few moments later based on:
 - How many records on the base table.
 - Whether the columns used for the:
 - ✓ **WHERE**
 - ✓ **SELECT** are indexed or not.
 - The type of index used (in case of indexing) → Primary index, Secondary/non-clustered index and/or Coverage Index
 - The capabilities of the used machine for the extraction process.

- **ONE** way to determine if the whole table download every time is a good choice or not is to roughly estimate the time needed for the extraction process.

- If the **WHOLE TABLE EVERY TIME EXTRACTION** is not possible and does not cope with the time associated to the process (time constraints/standards) and/or business requirements, then we extract the data **INCREMENTALLY (IF POSSIBLE, ALSO)**.
- Otherwise, the **WHOLE TABLE EVERY TIME** is used.
- Here the slowly changing dimensions and rapidly changing dimensions are determined and the way is determined.

DW Components

Extraction (E)

- **Incremental EXTRACT**

- If the table contains 2 million records, say it is the customer table, it probably takes 15- 20 Minutes (or more based on the answers to the above questions and the capability of the machine used) to download the whole table, if there are timestamp columns for creation and updates, then it is sensible data to try to download this table incrementally which reduces the download time to 3-5 minutes. Roughly the number of records decreases from 2 million records to 2000 (specific period changes ONLY E.g., Daily Changes ONLY).
- 5 Well known options to perform the incremental extract:
 - Using the Timestamp Columns.
 - Using Triggers.
 - Using the Transaction Date.
 - Using the Data Partitioning.
 - Mixed Approach (Combination of the above 4 ways).

DW Components

Extraction (E)

- **Incremental Extract Using Timestamp columns:**
 - **SELECT * FROM** DataSource1.Table **WHERE** DateColumn > **LSET**
 - **LSET** → Last Successful Extract Time.
 - Before the extract begins, the data warehouse system time is stored in a variable:
 - If the extract is **successful**, we store this variable into the DW control database (MetaData) as **LSET**, so it will be used to conduct next EXTRACT runs.
 - Else if the extract process **FAILs**, it is important to **NOT** store this variable into the DW control database (MetaData). So that **NEXT TIME WHEN THE** extract runs, it uses the **LSET** stored, and gets the records since the last successful extract.

DW Components

Extraction (E)

- **Incremental Extract Using Triggers:**
 - Triggers are defined ON the source system tables to keep the PKs (**Identifiers**) of the changed records (added and/or updated) in a dedicated different table **NAMED DELTA TABLE**.
 - The **DELTA TABLE** needs to be cleared out when it **SUCCESSFULLY** processed.

DW Components

Extraction (E)

- **Incremental Extract Using Transaction Date:**

- If the table processed and extracted is transactional table (contains dated transactions), it may contain more than one column to represent the date (this incremental method is valid to be applied here).
- Else if the table does not have a date BUSINESS columns, then the METADATA date columns may be used to conduct this purpose. E.g. Last Updated Date, Last Insertion Date.

DW Components

Extraction (E)

- **Incremental Extract Using Data Partitioning:**

- If a partitioning strategy is applied over the table downloaded or extracted, then this method is applicable, otherwise DW folks may recommend this to the operational systems folks.
- Using partitioning by range over time.
 - Extract records that fall within a specific time range (partition).
 - Partitioning data by range over time helps identifying the data for the current week.

DW Components

Transformation (T)

- **Once the data is extracted to the ODS/DSA, there are a set of protentional transformations, such as:**
 - Convert between units to match the target schema (DW Schema) used units → This serves the process of combining data from multiple sources.
 - Solve the duplicates (very complex process) especially for the **NON-Federated** data sources.
 - Standardize the formats (e.g. DateTime Formats).
 - Assign keys when integrating data from sources where **SOME OR ALL** of them are not federated.
 - Cleanse the data (Correct misspellings, resolving domain conflicts, **dealing with missing elements, ...**) [Fill in missing values, resolve noise data, handle the outliers especially if they represent noise data]
 - **Detect the duplicates/ conflicts.**
 - Apply data fusion.

DW Components

Transformation (T) → Cleanse (C)

■ Data quality problems appear over:

- **Single data source problems:**
 - ✓ Schema Problems (E.g. lack of integrity constraints, poor schema design,...)
 - ✓ Instance Problems (E.g. data entry errors)
- **Multiple data source problems (problems on integration process):**
 - ✓ Schema Problems (E.g. heterogeneous data models and schema design,...)
 - ✓ Instance Problems (E.g. different representations, different measurement system ...)

■ So data sources may conflict with each other over three levels:

- Schema Level Inconsistencies,
- Representation Level Inconsistencies, and
- Data Level Inconsistencies

DW Components

Transformation (T)

- ▶ Data quality problems after the integration process
- ▶ If we integrated complete and accurate data sources 2 main problems are appeared after the integration process:
 - **Mutual Inconsistency/Data Conflict:** multiple representations of same real-world object with conflicts between their data values (redundancy with different data values for the same object).
 - **Incompleteness:** Incomplete set as an answer to the user query (Less records than expected and/or existing or even MORE records than expected and/or existing)

DW Components

Transformation (T)

- **Three well defined strategies to handle conflicts:**
 - **Conflict Ignorance (CI) Strategy:** ignore handling with conflicts
 - **Conflict Avoidance (CA) Strategy :** Detect duplicates and avoid dealing with conflicts by taking a pre-decision to handle conflicts, this decision may be
 - Metadata based decision, or
 - Content-based decision.
 - **Conflict Resolution (CR) Strategy :** Detect duplicates and try to resolve them
 - Metadata based decision, or
 - Content-based decision

Conflict Handling Strategy/Step	Conflict Detection	Conflict Resolution
Conflict Ignorance	X	X
Conflict Avoidance	√	X
Conflict Resolution	√	√

DW Components

Transformation (T)

- Employees integrated from different data sources
- Extraction query gets ID, Name, Address, Zip_Code of employees from different data sources and applied the MAPPING required.
- And the data retrieved during the user query is:

ID	Name	Address	Zip_Code
1234567	Dandarawy	Xyz	1234
1234567	Dandarawy	Yxz	2341
112	Dandarawy	Xyz	1234

DW Components

Transformation (T)

ID	Name	Address	Zip_Code			
1234567	Dandarawy	Xyz	1234			
1234567	Dandarawy	Yxz	2341			
112	Dandarawy	Xyz	1234			
ID	Name	Address	Zip_Code	Source	Confidence	Timestamp
1234567	Dandarawy	Xyz	1234	S1	0.9	0.8
1234567	Dandarawy	Yxz	2341	S2	0.6	0.9

Data fusion is the process of combining these conflicting records into **COMPLETE & ACCURATE** Record. Instance based and MetaData Based Resolution/Fusion.

Extraction Query Q(Name, Address, Postal_Code, DoB)

**What is the data to be used when load the data to DW ???
T1, T2, or Both**

Islamic_Customers

ID	Name	Gender	Address	Postal_Code	DoB
1	Ali Zidane	1	XYFayoum	123456	11/8/1985
2	Aly Zedan	1	MNCairo	123458	NULL

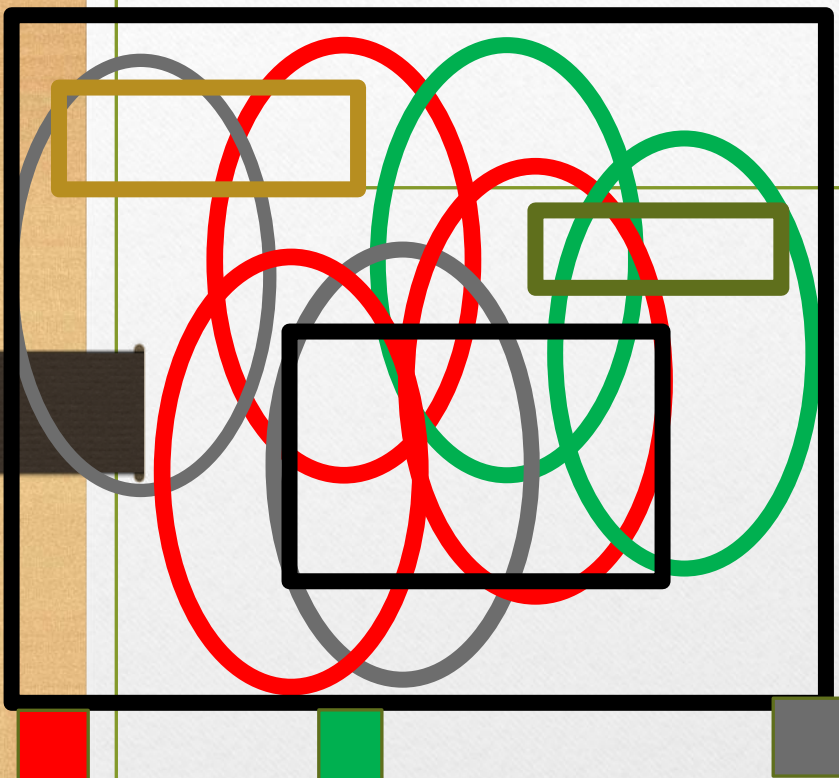
Customer

Islamic Customers

ID	Name	Gender	Address	Postal_Code	DoB	Type
1	Ali Zidane	1	XYFayoum	123456	11/8/1985	1
2	Ali Zidane	1	MNCairo	123458	11/8/19985	0

Customer

CID	Name	Gender	Address	Postal_Code	Type
2	Aly Zedan	1	MNCairo	123458	1
34	Noha Zidane	0	MMFayoum	345789	0



$O(X, Y, Z)$

$O_o(X, M, N, T)$

$Obj(Y, Z, N, T, F)$

- **Different Representations.**
- **Different Identifiers/MatcherSets.**
- **Different Mappings.**
- **Duplicated entities, and due to duplicates inconsistencies produced.**
- **Different lifetime states for the same duplicated real-world entity with inconsistent timestamps or even absent.**

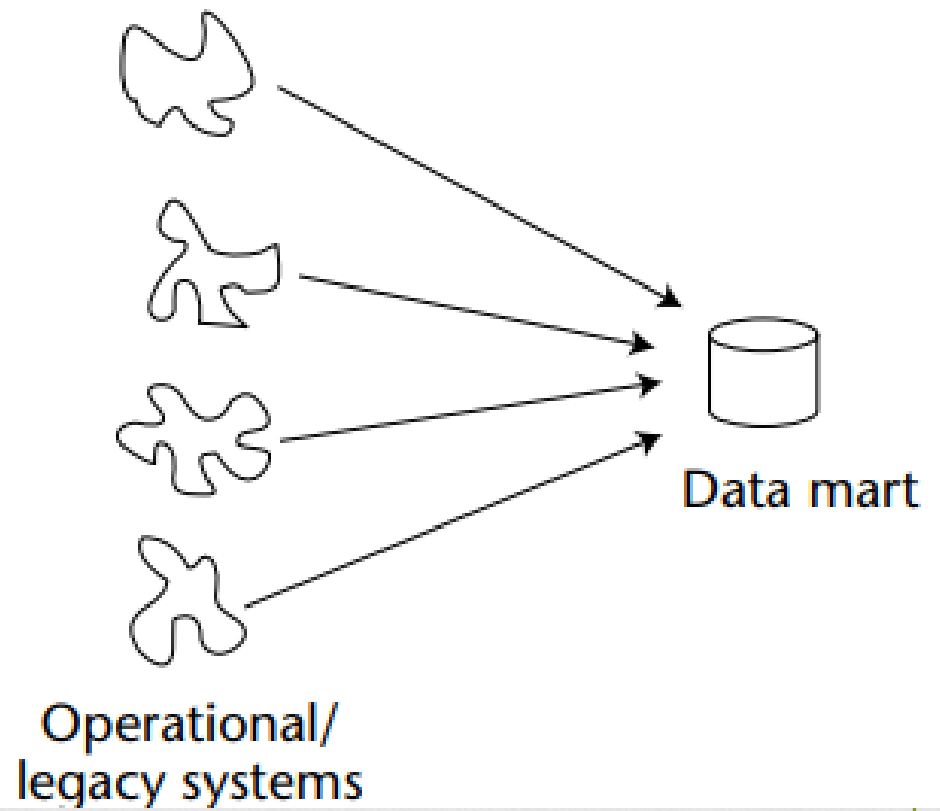
DW Components

Loading (L)

- ▶ Loading is the final step of the ETCL processes.
- ▶ Loading in the DW environment usually takes the form of presenting the quality assured data to the bulk/batch loading facilities of each data mart.
- ▶ The target data mart must then index the newly arrived data for query performance.

Independent Data Marts

The **Independent Data Mart** is a data mart that is built directly from the legacy applications.



Independent Data Marts

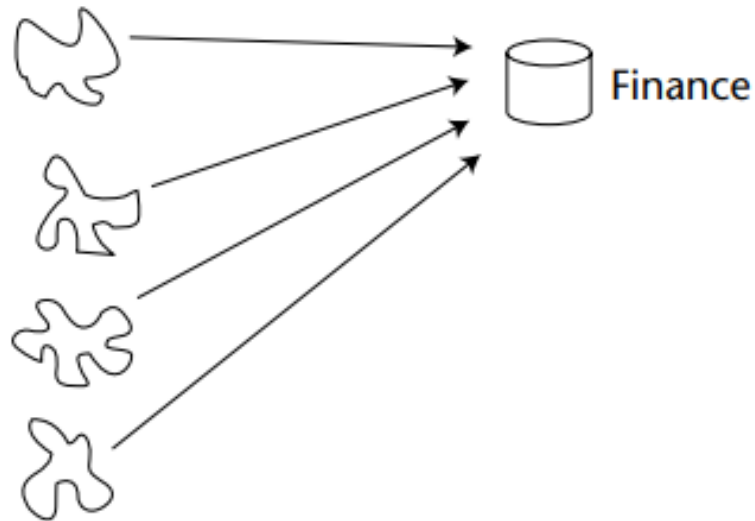
Independent data marts are good for short, fast solutions, but when the longer-term perspective is taken, Independent Data Marts are simply inadequate to solve the problems of information across the organization

Building Independent Data Marts: Pros. & Cons.

- Pros already detailed.
- Cons are shown below:

1. If there is only one data mart
→ There is no problem, but this is not always the case.

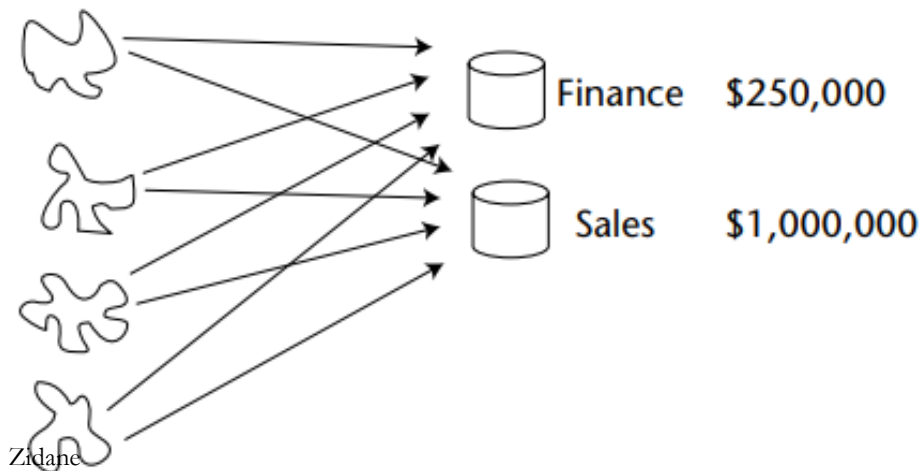
Zidane



The first independent data mart is successful.

Building Independent Data Marts: Pros. & Cons.

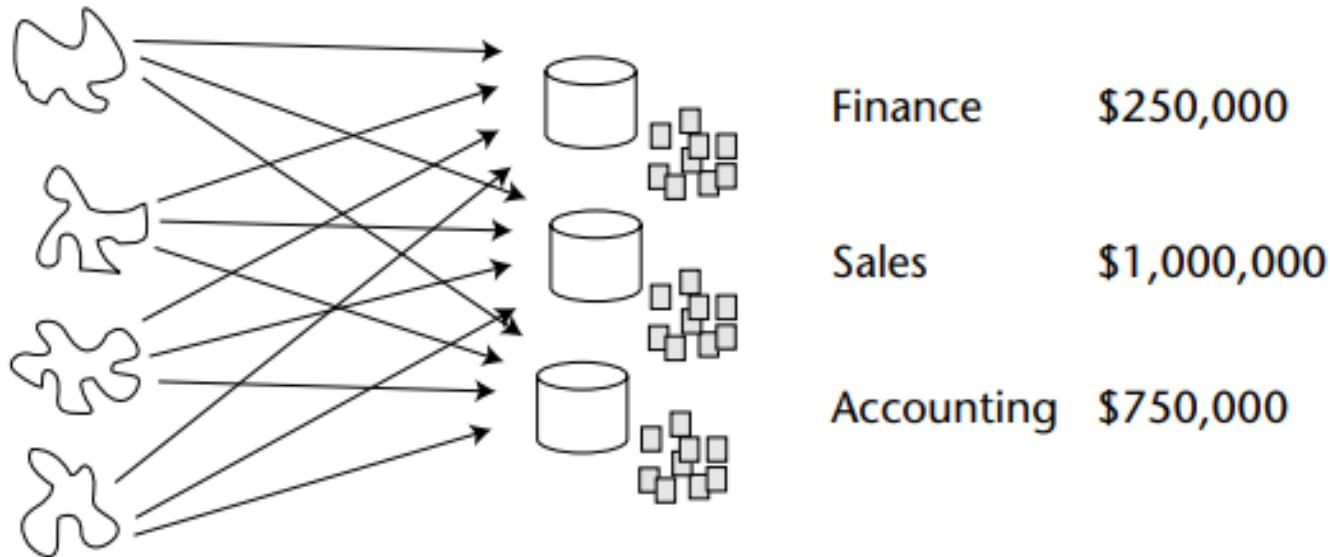
1. The Information between the two **Data Marts** is not Integrated and is not Synchronized.
2. **Same query to the two data marts give different answers.**



The second independent data mart is also successful. But someone notices that the answer to similar questions is not coordinated.

Building Independent Data Marts: Pros. & Cons.

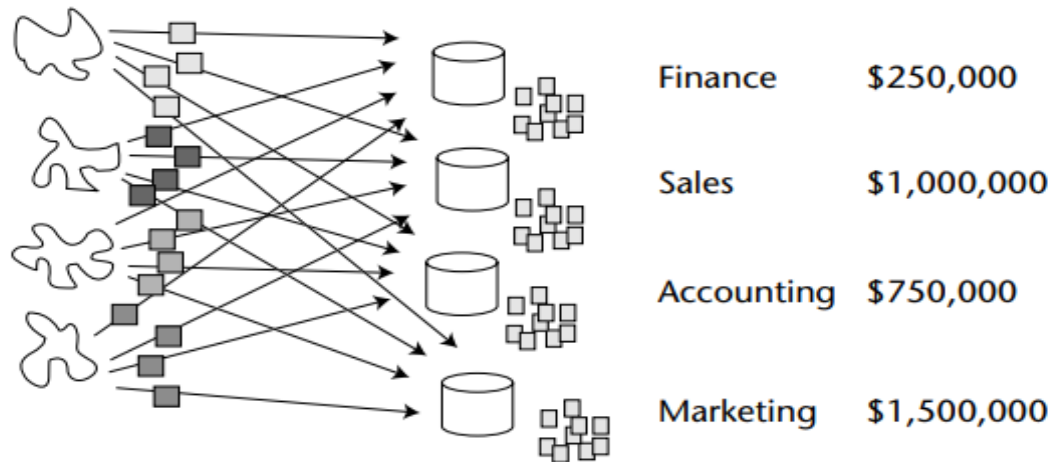
1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. **With each of the Independent Data Marts, there is an increasing redundancy of detailed data. ONE COPY of some or most of the data with each new data mart added (new business comes in).**



 Zidane The third data mart is added. Now there are three opinions on everything and the same detailed data appears every time a new data mart is added.

Building Independent Data Marts: Pros. & Cons.

1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. With each of the Independent Data Marts, there is an increasing redundancy of detailed data
4. **Too many interfaces and UI acting with data marts need to be built and maintained.**

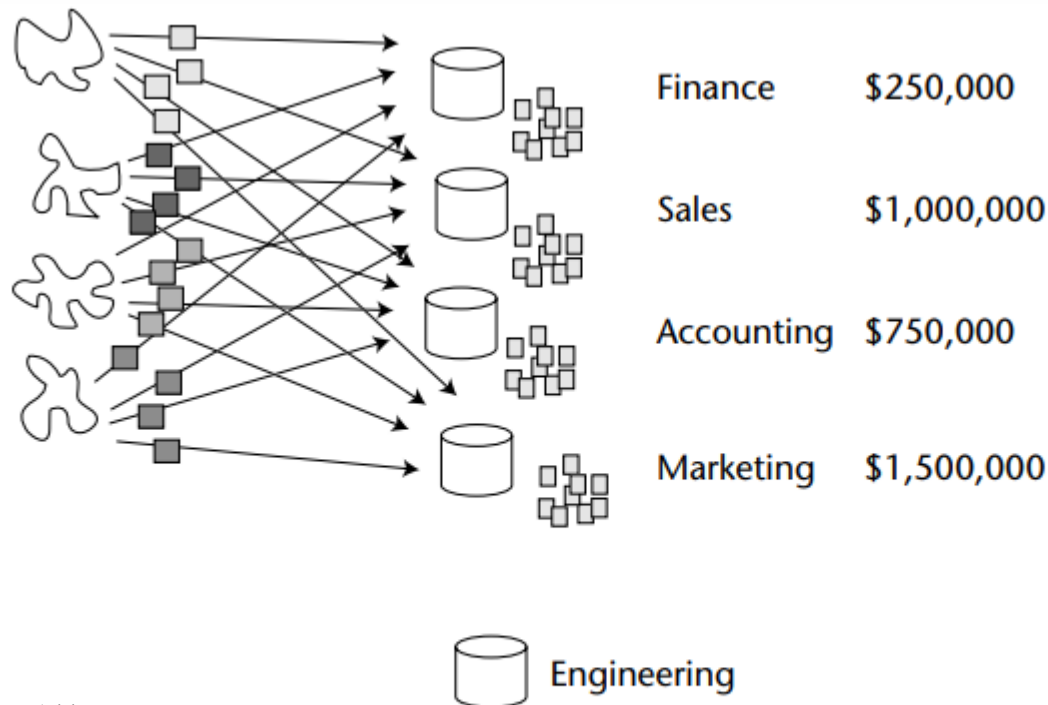


Zidane

The fourth data mart is added. There is yet one more option that cannot be correlated with any other opinion. There is more detailed data that is gathered and is redundant. There are a whole host of interface programs that have to be built and maintained.

Building Independent Data Marts: Pros. & Cons.

1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. With each of the Independent Data Marts, there is an increasing redundancy of detailed data.
4. Too many interfaces and UI acting with data marts need to be built and maintained.
5. **New data mart(s) redoes the same work or very similar one more time which increases the other cons. Especially if none of the ready components are reusable**

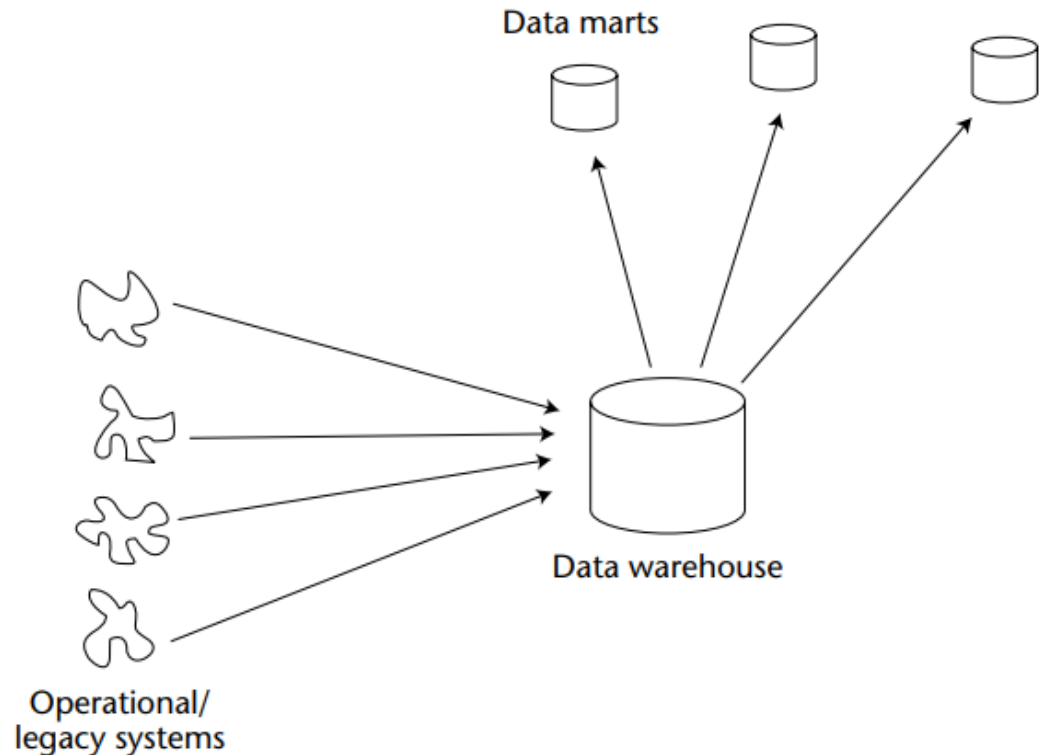


Zidane

Engineering wishes to build a data mart. There is no foundation on which to build and there is no historical data to access. All integration must be redone (one more time), and there is no coordination with any previous efforts at integration.

Dependent Data Marts

The opposite of an Independent Data Mart is a Dependent Data Mart. A Dependent Data Mart is one that is built from data coming from the data warehouse.



Dependent Data Marts are simply adequate to solve the problems of information across the organization

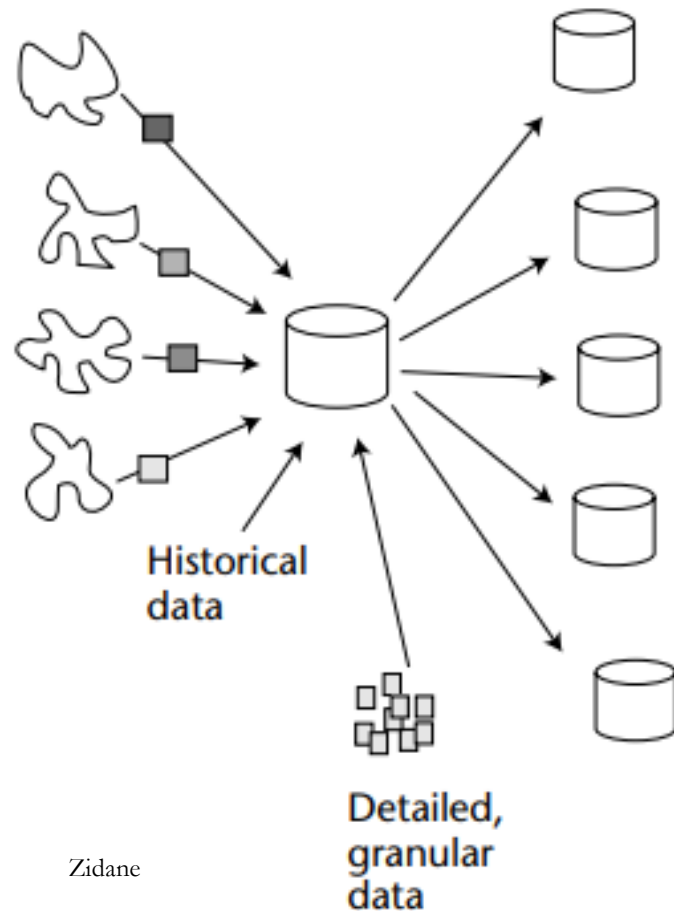
Zidane

- The **Dependent Data Mart** does not depend on legacy or operational data for its source.
- **Dependent Data Mart** depends on only the data warehouse for its source data.
- The **Dependent Data Mart** requires forethought and investment. It requires that someone “think globally.”
- The **Dependent Data Mart** requires multiple users to pool their information needs for the creation of the data warehouse.
- In other words, the **Dependent Data Mart** requires advance planning, a long-term perspective, global analysis, and cooperation and coordination of the definition of requirements among different departments of an organization.

What are Dependent Data Marts

Dependent Data Marts

With dependent data marts, all problems of independent data marts are solved.



Zidane

With dependent data marts, all problems of independent data marts are solved.

To Summarize ...

- The main difference between **Dependent & Independent Data Marts** is that the **Dependent Data Marts** get data from an already created data warehouse while the **Independent Data Marts** get data directly from an operational source and/or external source.

To Summarize ...

- In brief, a data warehouse is a system that helps to analyze data, create reports and visualize them to make business decisions
- An organization has multiple data sources
- Data warehouses collect this data and help to analyze them to take business insights.
- A **Data Mart** is a subset of data stored in a data warehouse. These **Data Marts** provide more security and data integrity.
- Furthermore, there are two types of data marts as **Dependent & Independent Data Marts**.

To Summarize ...

The problems with the **Independent Data Marts** are that the independent data marts

1. Do not provide a platform for reusability
2. Do not provide a basis for reconciliation of data
3. Do not provide a basis for a single set of legacy interface programs
4. Do require that every **Independent Data Mart** create its own pool of detailed data, which is, unfortunately, massively redundant with the pools of detailed data created by other **Independent Data Marts**.
5. The good thing is that; **Dependent Data Marts** that take data from a data warehouse do not have to have the same set of architectural problems.

To Summarize ...

- **Dependent VS Independent Data Marts**
 - **Methods of obtaining data:** **Dependent Data Marts** get data from a central data warehouse that has already been created. However, **Independent Data Marts** get data directly from operational sources or external sources of data or both.
 - **Functionality:** **Dependent Data Marts** obtain data by aggregating, restricting and summarizing the data of the data warehouse **Independent Data Marts** get data from multiple transaction systems in one subject area or department to support specific business needs. Hence, this is a major difference between **Dependent & Independent Data Marts**.
 - **Dependability:** **Dependent Data Marts** depend on the central data warehouse while **Independent Data Marts** are standalone data marts and do not depend on the central data warehouse.
 - **Security:** Another difference between **Dependent & Independent Data Marts** is that the **Independent Data Marts** are more secure than **Dependent Data Marts**.

To Summarize ...

- **Dependent VS Independent Data Marts**
 - Control on Data: One major difference between **Dependent & Independent Data Marts** based on control. **Independent Data Marts** have more control over their data than **Dependent Data Marts**.

Conclusion:

- In brief, a data mart is a subset of a data warehouse that focuses on a single functional area or an organization.
- There are two types of data marts as **Dependent & Independent Data Marts**.
- The difference between **Dependent & Independent Data Marts** is that the **Dependent Data Marts** get data from already created data warehouse while the independent data marts get data directly from an operational, external source or both.

Hybrid Data Marts

Hybrid Data Marts

- A **Hybrid Data Mart** combines data from an existing data warehouse and other operational source systems.
- It unites the speed and end-user focus of a top-down approach with the benefits of the enterprise-level integration of the bottom-up method.